

Le langage procédural en SQL sous PostGres

Les curseurs

PL/pgSQL
(Procedural Language/postgreSQL Structured Query Language)

A decorative graphic on the left side of the slide, consisting of a vertical arrangement of blue and yellow hexagons of various sizes, some solid and some outlined, creating a pixelated or molecular-like structure.

Les curseurs

■ Principe :

- Structure du langage qui permet de manipuler les résultats de requêtes ligne par ligne;
- Il s'agit de préparer des requêtes qui seront ensuite exécutées et dont on pourra parcourir le résultat dans une boucle ;
- Ils sont un palliatif à la Structure FOR .. IN, la seule différence c'est que la requête est "stockée" , et réutilisable à tous moments

Les curseurs

■ Déclaration d'un curseur

DECLARE

`curseur1` *refcursor*; -- à définir lors de l'ouverture

`curseur2` CURSOR FOR SELECT ...;

`curseur3` CURSOR(*key integer*) IS SELECT ... WHERE *id=key*;

- Ces variables sont toutes du type de données *refcursor*, mais la première peut être utilisées avec n'importe quelle requête, alors que la seconde a une requête complètement spécifiée qui lui est déjà liée, et la dernière est liée à une requête paramétrée. (*key* sera remplacée par un paramètre de valeur entière lors de l'ouverture du curseur.)
- La variable `curseur1` est dite *non liée* puisqu'elle n'est pas liée à une requête particulière, il faudra la définir lors de l'ouverture du curseur
- Les curseurs associés à une requête sont qualifiés de curseur lié (*bound_cursor*)

Les curseurs

■ Ouverture et fermeture d'un curseur

- Avant toute utilisation le curseur doit être ouvert
 - Ouverture d'un curseur lié (*ouvre et exécute la requête associée*):

```
OPEN Curseur2 ;  
OPEN Curseur3 (42) ;
```
 - Ouverture d'un curseur NON lié : *il s'agit de lui associer une requête lors de l'ouverture*
 - *OPEN ... FOR SELECT*

```
OPEN Curseur1 FOR SELECT * FROM foo WHERE key = p_ykey;
```
 - *OPEN ... FOR EXECUTE*

```
OPEN Curseur1 FOR EXECUTE 'SELECT * FROM ' || quote_ident($1) ;
```
- Après utilisation le curseur doit être fermé

```
CLOSE Curseur1 ;  
CLOSE Curseur2 ;  
CLOSE Curseur3 ;
```

Les curseurs

■ Utilisation d'un curseur

Une fois qu'un curseur a été ouvert, il peut être manipulé grâce à l'instruction suivante :

```
FETCH curseur1 INTO cible;
```

FETCH retourne la ligne courante du résultat du curseur dans la cible, qui peut être une variable ligne, une variable record ou une liste de simples variables séparées d'une virgule, exactement comme SELECT INTO.

Comme pour SELECT INTO ou le PERFORME, la variable spéciale FOUND peut être vérifiée pour voir si une ligne a été obtenue ou pas.

■ Exemple :

```
FETCH curseur1 INTO var_ligne;  
FETCH curseur2 INTO foo, bar, baz;
```

Il existe d'autres paramètres à l'instruction fetch

Les curseurs

■ Exemple d'utilisation d'un curseur

```
CREATE or replace FUNCTION copy_form_curseur(p_id int) RETURNS void AS $$
DECLARE
    use_form formateur%ROWTYPE;
    -- déclaration d'un curseur paramétré
    curs1 CURSOR (key integer) for SELECT * FROM formateur WHERE id_formateur < key;
BEGIN
    EXECUTE 'drop TABLE IF EXISTS temp';
    EXECUTE 'CREATE TABLE temp (id_f integer, nom varchar(50))';
    OPEN curs1 (p_id); -- ouverture du curseur paramétré
    LOOP -- parcours du curseur paramétré
        FETCH curs1 INTO use_form;
        EXIT WHEN NOT FOUND; -- sort de la boucle si le curseur ne retourne plus rien
        EXECUTE 'INSERT INTO temp VALUES(' || use_form.id_formateur || ', ' ||
            quote_literal(use_form.nom_formateur) || ')';
    END LOOP;
    Close curs1;
END; $$ LANGUAGE plpgsql;
```



FIN